

Section: XSS(Stored), JavaScript, and Command Injection

Introduction:

This part focuses on analyzing selected vulnerabilities in DVWA, including XSS(Stored), JavaScript, and Command Injection across different security levels (Low, Medium, High).

XSS (Stored Cross Site Scripting)

Low Level:

At Low security level, the application does not implement any input sanitization or output encoding, making it vulnerable to Cross-Site Scripting (XSS) attacks.

Steps:

1. Navigate to the XSS page.
2. Enter a malicious JavaScript payload in the input field (e.g., `<script>alert('XSS')</script>..`)
3. Submit the input.
4. Capture the request using Burp Suite.

Result:

The injected script was executed successfully in the browser.

Analysis:

The application does not validate or sanitize user input before displaying it back to users, allowing malicious scripts to execute in the client's browser and making it vulnerable to XSS attacks.



Figure 1: DVWA Security Level set to Low

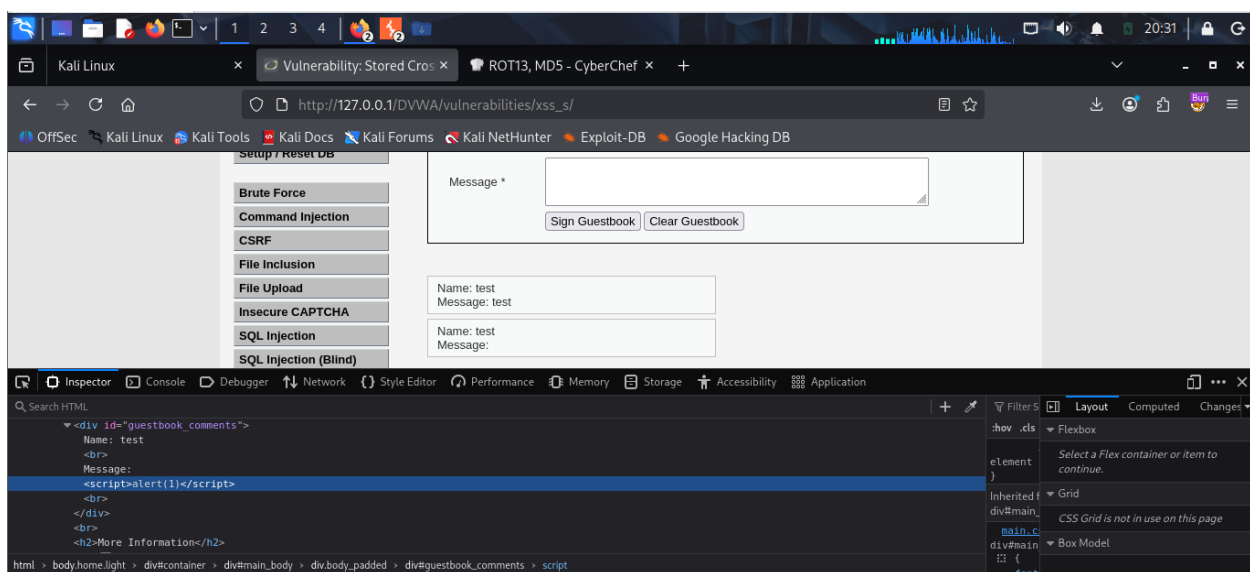


Figure 2: XSS Stored Low – Injected script displayed in the page source

Medium Level:

At Medium security level, the application introduces partial protection by filtering specific HTML tags and restricting basic script injection attempts.

Steps:

1. Change security level to Medium.
2. Navigate to the XSS page.
3. Enter a JavaScript payload and submit the request.
4. Capture and inspect the request using Burp Suite.
5. Observe the application response.

Result:

Basic script payloads were blocked or filtered, but alternative payload techniques may still execute successfully.

Analysis:

Although the application applies input filtering to reduce XSS attacks, the protection is incomplete and can often be bypassed using encoded payloads or alternative event handlers. This makes the application still vulnerable to Cross-Site Scripting (XSS) attacks.

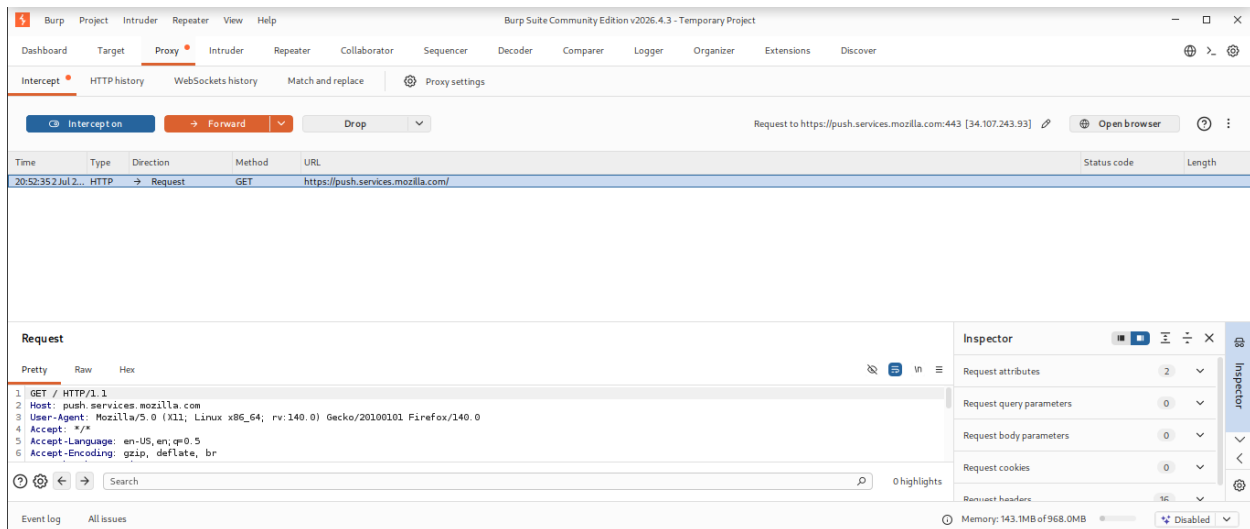


Figure 3: XSS Stored Medium – Request intercepted using Burp Suite.

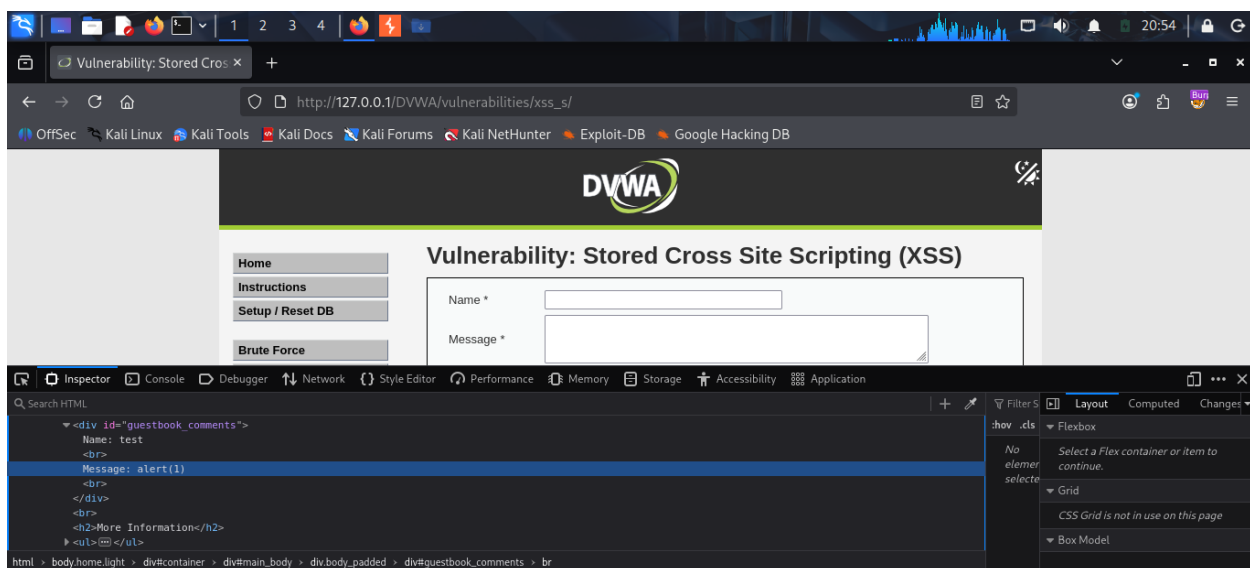


Figure 4: XSS Stored Medium – Injected XSS payload displayed in the HTML source.

High Level:

At High security level, the application applies stronger input filtering and sanitization mechanisms to reduce the possibility of Cross-Site Scripting (XSS) attacks.

Steps:

1. Change security level to High.
2. Navigate to the XSS page.
3. Enter an XSS payload in the input field.
4. Capture and inspect the request using Burp Suite.

Result:

The application filtered or blocked the malicious script, preventing it from executing in the browser.

Analysis:

The application performs stronger validation and sanitization of user input before rendering it in the response. This reduces the effectiveness of common XSS payloads and improves protection against Cross-Site Scripting attacks.

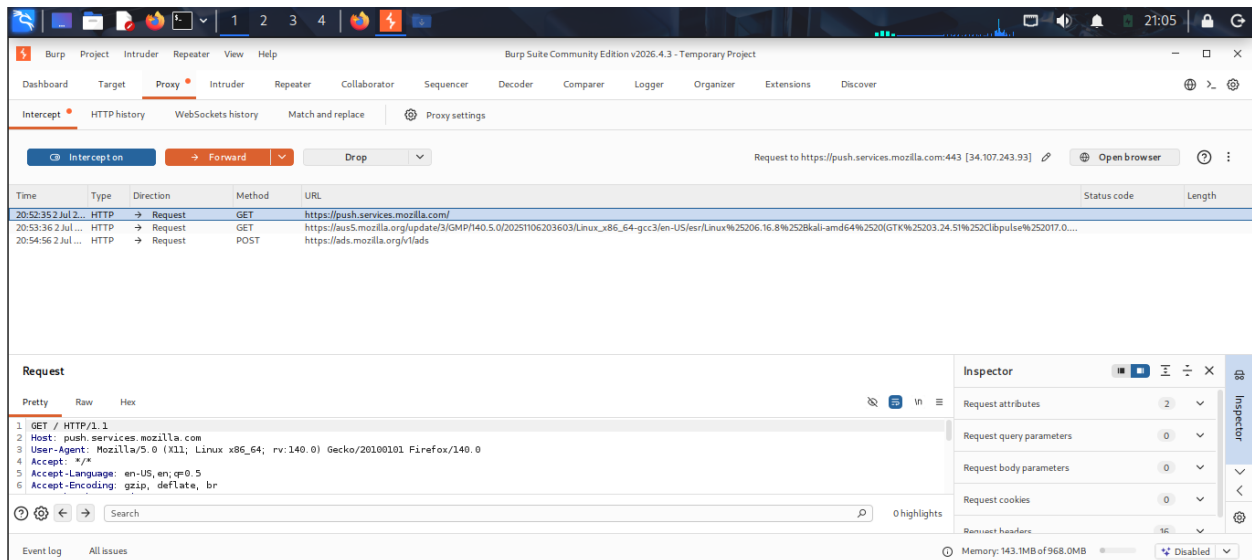


Figure 5: XSS Stored Medium – Request intercepted using Burp Suite.

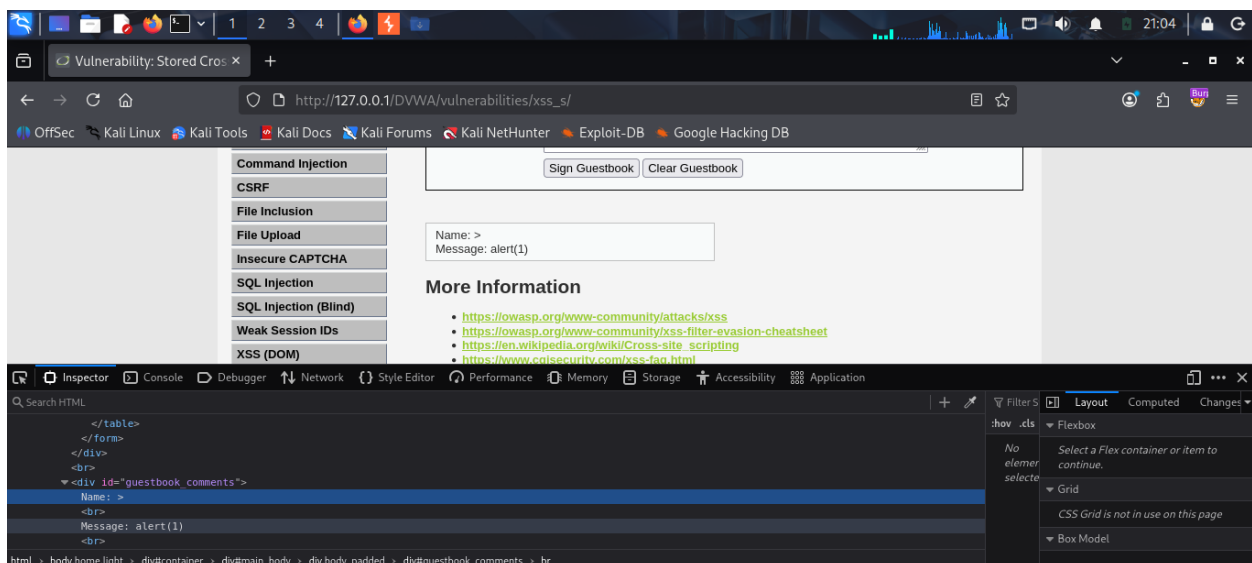


Figure 6: XSS Stored Medium – JavaScript payload executed successfully.

JavaScript

Low Level:

At Low security level, the application does not apply client-side validation or restrictions on JavaScript input handling.

Steps:

1. Change security level to Low
2. Navigate to the JavaScript page in DVWA.
3. Interact with the application and submit the input.
4. Observe the application behavior.

Result:

The request was processed without additional validation or restrictions.

Analysis:

The application relies on minimal client-side controls and does not enforce strict validation mechanisms, allowing user-controlled input to be processed directly.



Figure 7: phrase entered successfully after we changed the token

Medium Level:

At Medium security level, the application introduces basic client-side validation using JavaScript before processing requests.

Steps:

1. Change security level to Medium.
2. Navigate to the JavaScript page.
3. Submit the input normally.
4. Capture and inspect the request.
5. Analyze the transmitted parameters.

Result:

The application performed client-side checks before sending the request.

Analysis:

Basic JavaScript validation was implemented to reduce invalid input submission. However, Burp Suite demonstrated that client-side validation alone is insufficient because requests can still be modified manually.

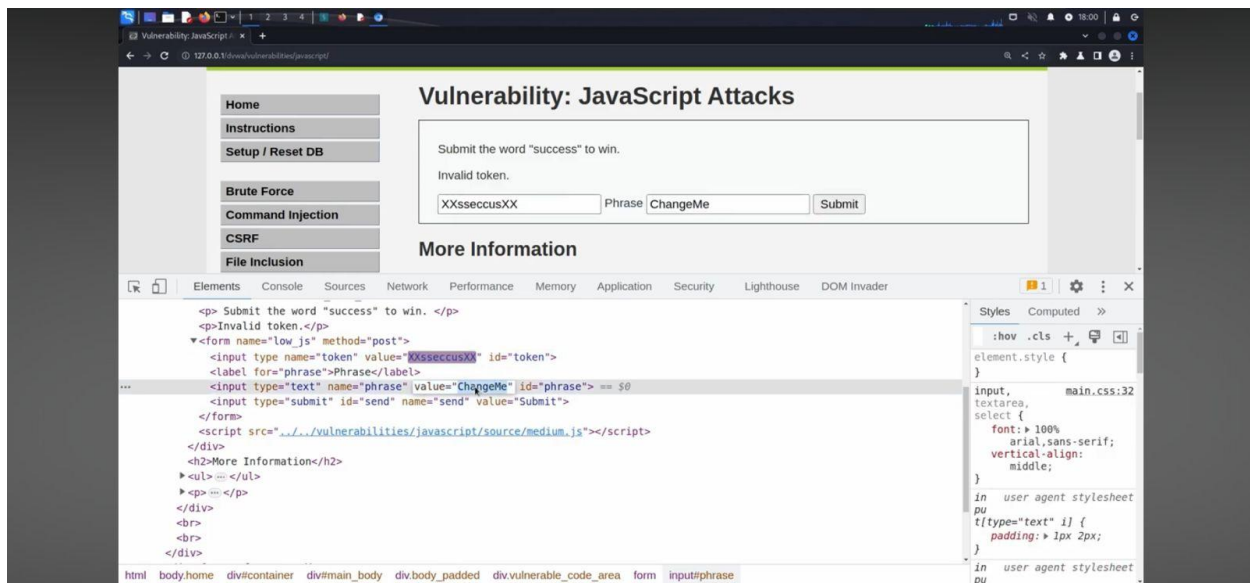


Figure8: JavaScript Attacks Medium – Page source showing the token and phrase values.

High Level:

At High security level, the application combines stronger validation mechanisms to improve request handling.

Steps:

1. Change security level to High and repeat the last steps.

Result:

Additional validation controls were applied before request processing.

Analysis:

The application strengthens validation and reduces reliance on client-side JavaScript checks. Burp Suite analysis showed improved request handling compared to lower security levels.

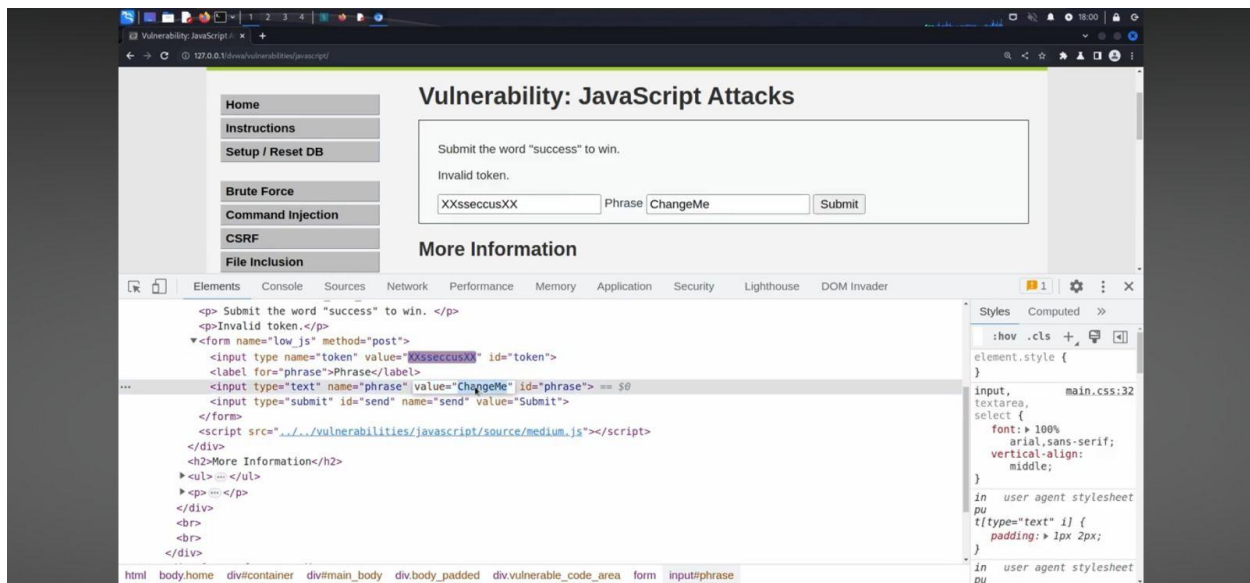


Figure 9: JavaScript Attacks High – Obfuscated JavaScript source code

Command Injection

Description:

Command Injection is a security vulnerability that allows attackers to execute unauthorized system commands due to improper input validation.

Low Level:

At Low security level, the application does not validate user input before executing system commands.

Steps:

1. Change security level to Low
2. Navigate to the Command Injection page in DVWA.
3. Enter the payload and capture the request.

Result:

The injected command was executed successfully.

Analysis:

The application executes user input without proper validation, making it vulnerable to Command Injection attacks.

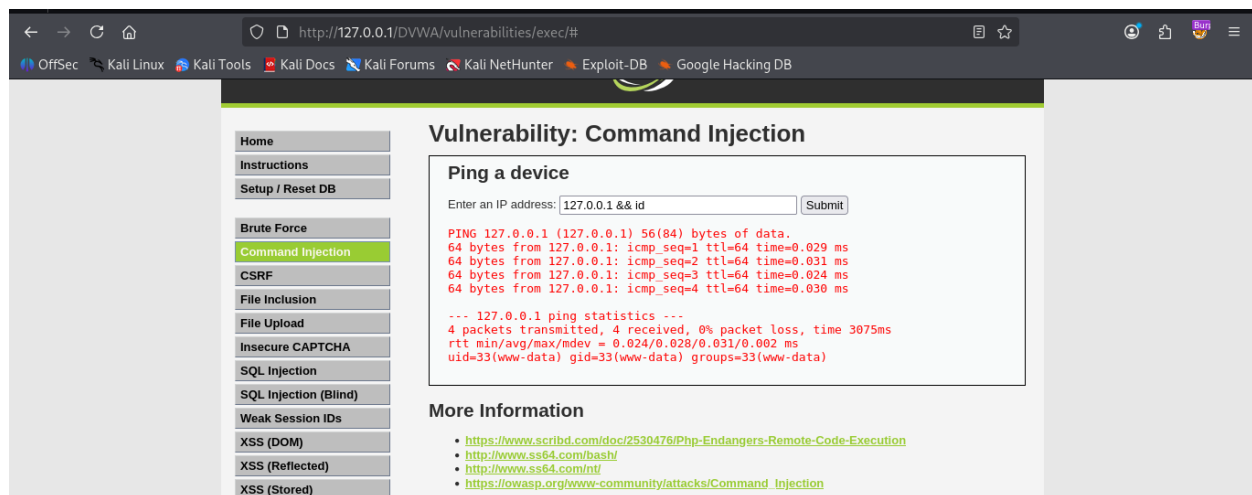


Figure 10: Command Injection Low – Successful execution of the injected id command.

Medium Level:

At Medium security level, the application applies basic input filtering to reduce command injection attempts.

Steps:

1. Change security level to Medium
2. Submit the payload and inspect the request.

Result:

Some malicious characters were filtered before execution..

Analysis:

The application performs basic filtering, but the protection is incomplete and may be bypassed using alternative payloads

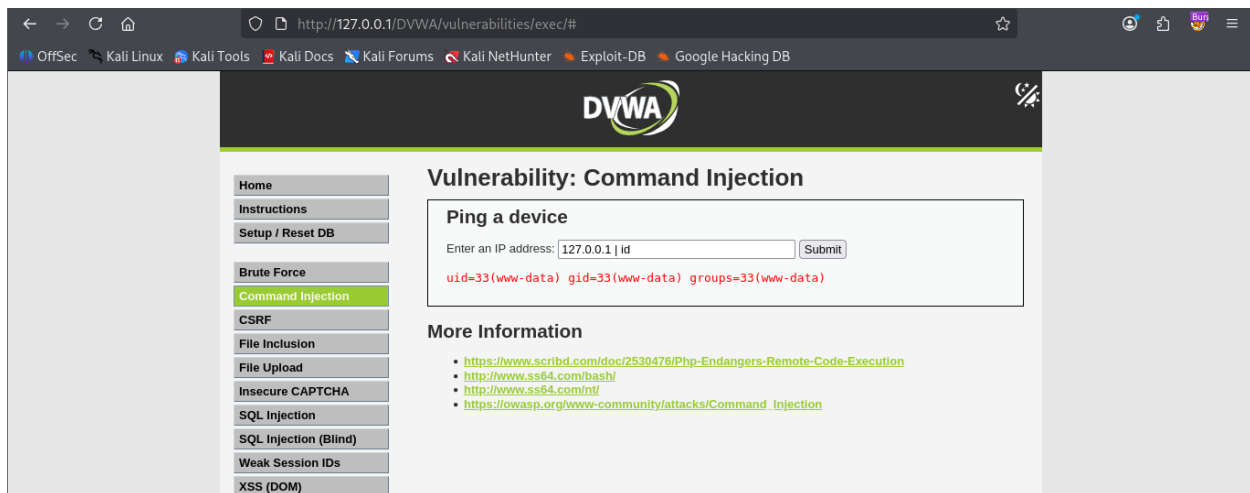


Figure 11: Command Injection Medium – Command executed successfully after bypassing input filtering.

High Level:

At High security level, the application implements stronger validation to prevent command injection.

Steps:

1. Change security level to High.
2. Submit the payload and inspect the request.

Result:

The malicious command was blocked and was not executed.

Analysis:

The application applies stronger input validation, preventing malicious commands from being executed.

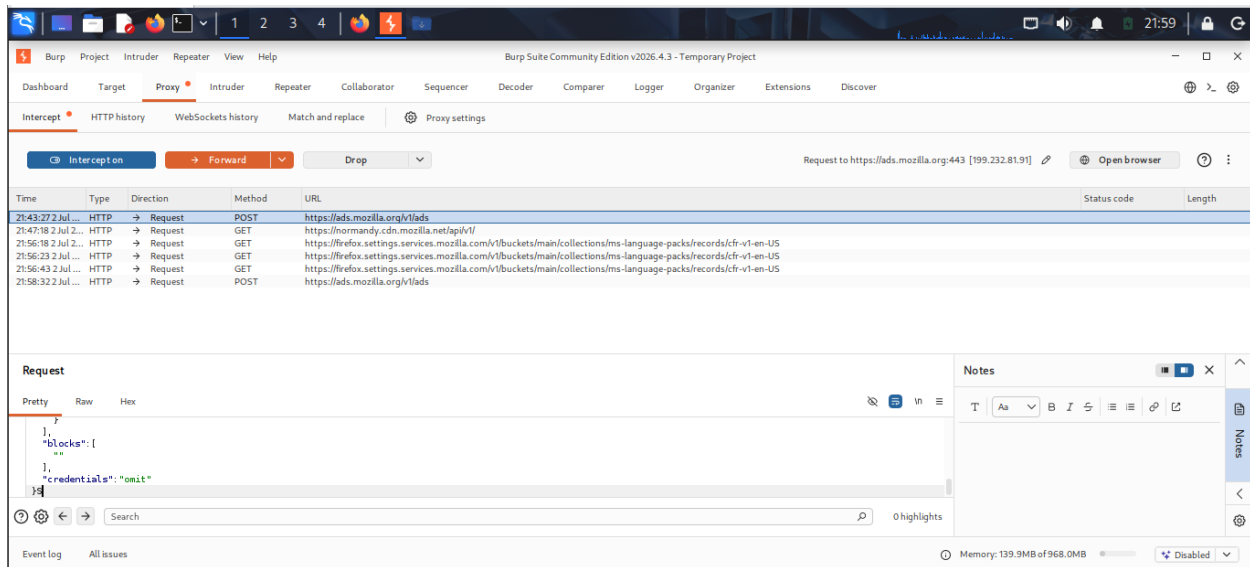


Figure12: Command Injection High – Request intercepted using Burp Suite.

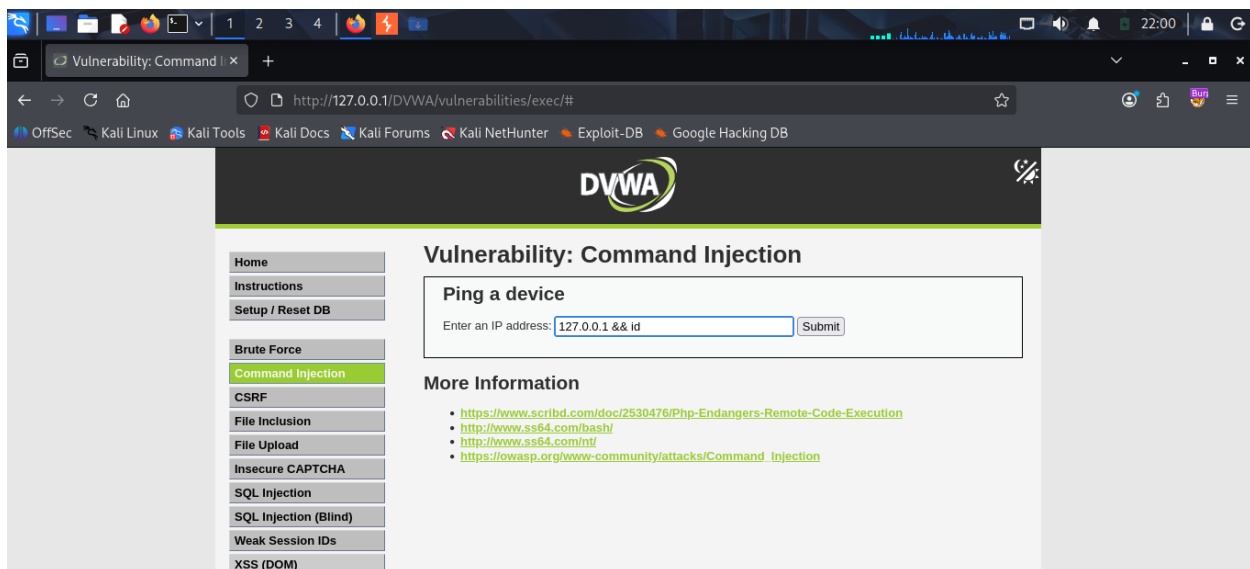


Figure13: Command Injection High – Malicious input blocked by the application.